Лабораторная работа №3
Применение статических анализаторов программного кода

1 Цель работы

1.1 Изучить процесс использования статических анализаторов кода для повышения качества программного обеспечения.

2 Литература

2.1 Зверева В. П., Сопровождение и обслуживание программного обеспечения компьютерных систем : учебник для студ. учреждений сред. проф. Образования / В. П. Зверева, А. В. Назаров. – М. : Издательский центр «Академия», 2021. – 256 с.

3 Подготовка к работе

3.1 Повторить теоретический материал по статическому анализу программного кода.

3.2 Изучить описание лабораторной работы.

4 Основное оборудование

4.1 Персональный компьютер с Visual Studio, средствами разработки на C# и доступом к NuGet.

5 Задание

5.1 Установка Roslynator и Meziantou.Analyzer

5.1.1 Открыть Visual Studio.

5.1.2 Создать новый проект C# или открыть существующий.

5.1.3 В обозревателе решений нажать правой кнопкой мыши на проект и выбрать «Управление пакетами NuGet».

5.1.4 На вкладке «Обзор» выбрать источник nuget.org, найти пакет Roslynator.Analyzers и установить совместимую с проектом и средой разработки версию.

5.1.5 Аналогичным образом найти и установить пакет Meziantou.Analyzer.

5.1.6 Дождаться восстановления пакетов и выполнить сборку проекта. В решении с несколькими проектами подключить анализаторы к каждому проверяемому проекту.

5.2 7. Зафиксировать в отчете версии Visual Studio, .NET SDK и установленных пакетов.

5.2.1 8. При необходимости добавить в учебный проект примеры нарушений правил из документации анализаторов: избыточные конструкции кода, сравнение строк без явного StringComparison, создание пустого массива вместо Array.Empty<T>(). Убедиться, что проект компилируется.

5.3 Анализ кода

5.3.1 Открыть окно «Список ошибок» и включить отображение ошибок, предупреждений и сообщений. Для источника диагностик выбрать сборку и IntelliSense.

5.3.2 Выполнить повторную сборку проекта и изучить диагностики Roslynator (идентификаторы RCS…) и Meziantou.Analyzer (идентификаторы MA…). Отделить их от сообщений компилятора и встроенных анализаторов .NET.

5.3.3 Для каждого обнаруженного нарушения изучить описание правила, место в коде и предлагаемый способ исправления. Проверить, сохраняется ли поведение программы.

5.3.4 Исправить выявленные проблемы, повторно выполнить сборку и сравнить результаты до и после исправлений. Сохранить примеры кода и снимки для отчета. Если ожидаемое сообщение отсутствует, проверить настройки правила и соответствие примера его условиям.

6 Порядок выполнения работы

6.1 Повторить теоретический материал п.2.

6.2 Выполнить установку Roslynator и Meziantou.Analyzer и анализ кода п.5.1-5.3

6.3 Ответить на контрольные вопросы п.8.

6.4 Оформить отчет в соответствии с разделом п.7.

7 Содержание отчета

7.1 Титульный лист;

7.2 Цель работы;

7.3 Результаты анализа Roslynator и Meziantou.Analyzer: идентификаторы и описания диагностик, примеры кода до и после исправлений, снимки экрана и сравнение результатов повторного анализа;

7.4 Ответы на контрольные вопросы;

7.5 Вывод по проделанной работе.

8 Контрольные вопросы

8.1 Что такое статический анализ кода и в чем его основные преимущества?

8.2 Какие основные отличия между статическим и динамическим анализом кода?

8.3 Какие типы проблем могут обнаружить Roslynator и Meziantou.Analyzer в коде C#?

8.4 Какие типы ошибок могут быть пропущены статическими анализаторами?

9 Приложение

Roslynator – набор инструментов на основе Roslyn для анализа и улучшения кода C#. В этой работе используется пакет Roslynator.Analyzers, который выявляет нарушения правил и предлагает улучшения кода.

Meziantou.Analyzer – набор анализаторов C# на основе Roslyn, выявляющих потенциальные ошибки, проблемы использования API и

производительности, а также нарушения рекомендуемых практик. Для части диагностик доступны автоматические исправления.

Оба пакета подключаются к проекту через NuGet. Диагностики отображаются в Visual Studio и при сборке в зависимости от настроек правил. Важность отдельных диагностик настраивается через .editorconfig. Статический анализ дополняет тестирование и не гарантирует обнаружение всех ошибок.

Документация Roslynator: https://github.com/dotnet/roslynator

Документация Meziantou.Analyzer: https://github.com/meziantou/Meziantou.Analyzer